

UNIDAD N° 3

VISUALIZACIÓN DE DATOS

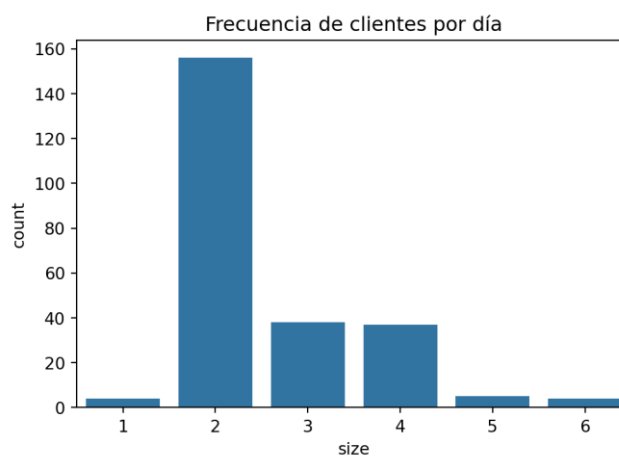
Marcas visuales y elementos preatentivos.
Tipos de gráficos y diagramas.
Visualización de datos como forma de comunicar.

UNIDAD N° 3: GRÁFICOS Y DIAGRAMAS

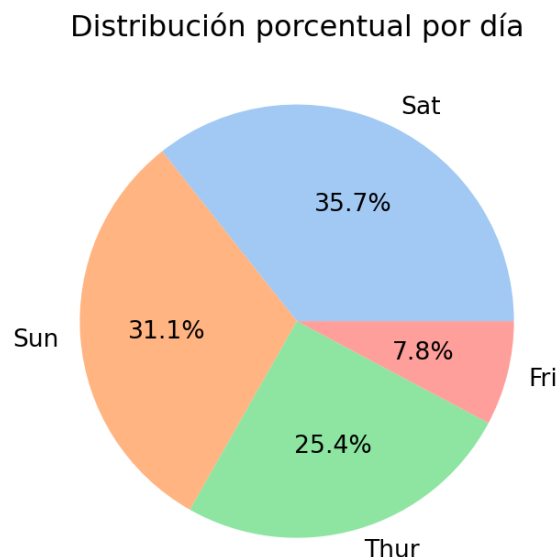
1. GRÁFICOS PARA VISUALIZAR DATOS

1.1 Gráficos de una sola variable

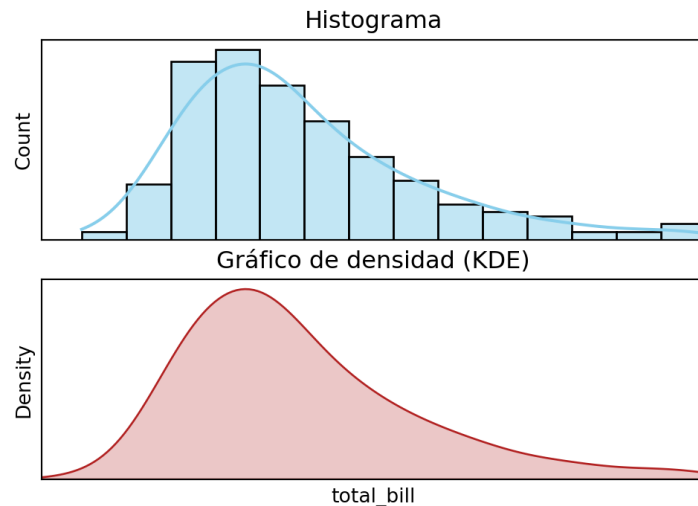
- **Gráfico de barra.** Es el gráfico más utilizado y el más efectivo para comunicar información. Puede representar tanto variables cualitativas como cuantitativas.



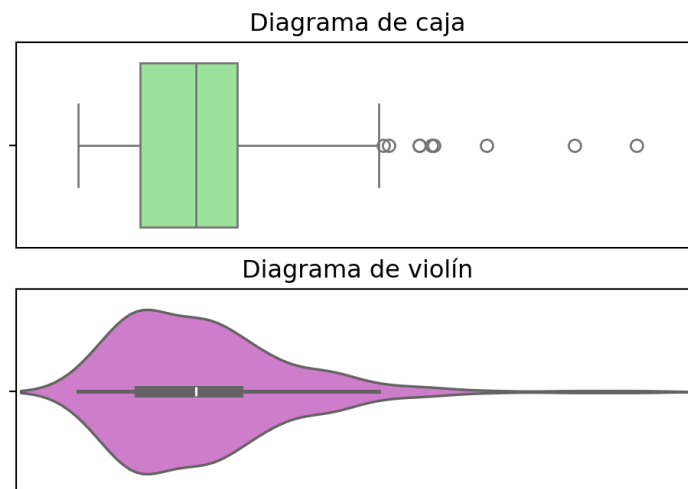
- **Gráfico de torta.** Otro de los gráficos más utilizados. Se usa especialmente para indicar un valor numérico (por ejemplo, proporciones y porcentajes) asociado a una variable cualitativa. Aunque es fácil de entender este tipo de gráficas, es difícil en lo visual comprender los datos representados. Por eso muchos especialistas critican su uso.



- **Histograma / polígono de frecuencias.** Gráfico útil para representar variables cuantitativas continuas. Permite ver la distribución de valores de una variable en un intervalo dado.
- **Gráficos de densidad (KDE).** Similar a un histograma, pero con curvas suavizadas.

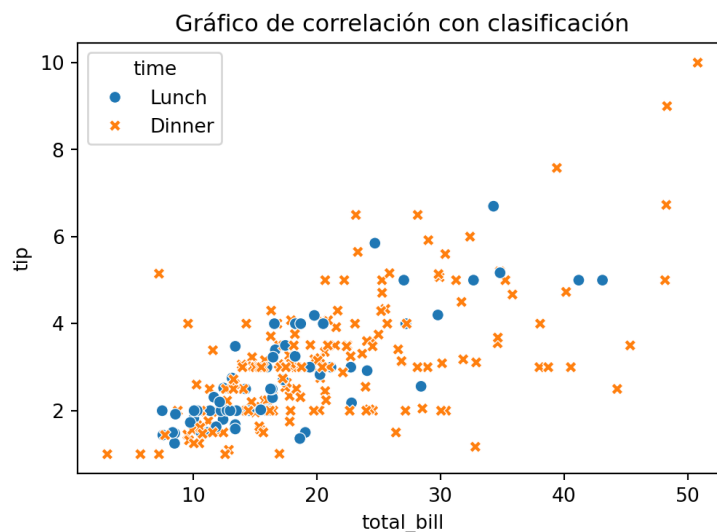


- **Diagramas de caja.** Diagrama de gran utilidad para ver en un vistazo los estadísticos básicos de un conjunto. Sirve para variables cuantitativas, principalmente continuas.
- **Diagramas de violín.** Es una mezcla entre el diagrama de caja y el diagrama de densidad. No solo se ven los valores estadísticos del conjunto, sino también la distribución fina de los valores.



1.2 Gráficos para muchas variables

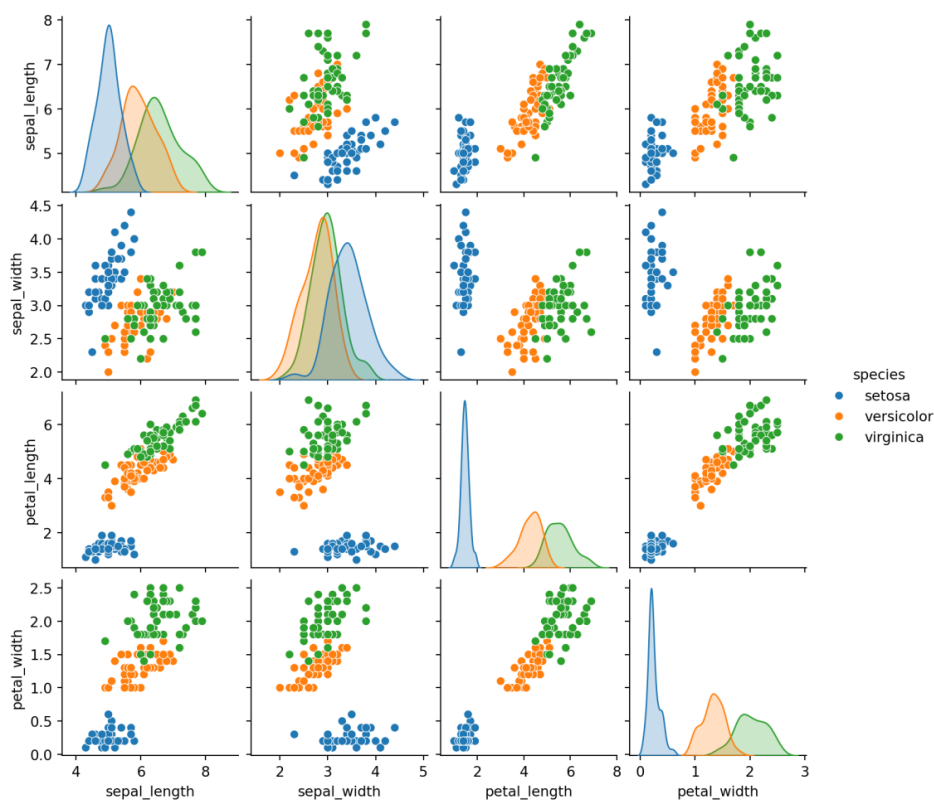
- **Diagrama de dispersión.** Visualiza dos variables cuantitativas continuas. Resulta muy útil para ver correlaciones y tendencias. Por ejemplo, en el siguiente ejemplo se muestra la dispersión de puntos (posición) que representan dos variables cuantitativas (total_bill y tip), mientras la forma de los puntos cambian de acuerdo a una variable categórica (time).



- **Gráfico de mosaicos (treemap).** Adecuado cuando se tienen una o más variables cualitativas y una variable cuantitativa, principalmente si esta última representa la parte de un total.

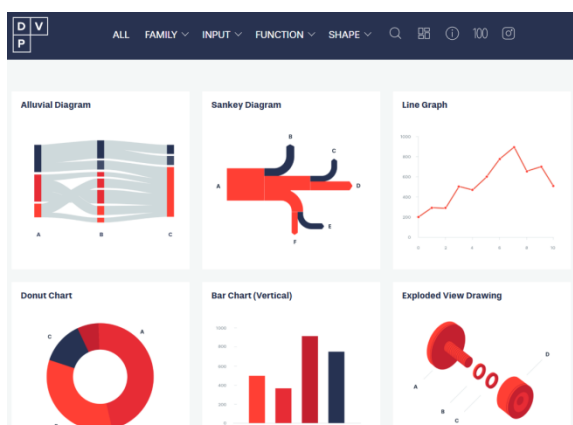


- **Matriz de gráficas de dispersión.** Conjunto de gráficos que permite ver el comportamiento de las variables de forma individual y bivariada.

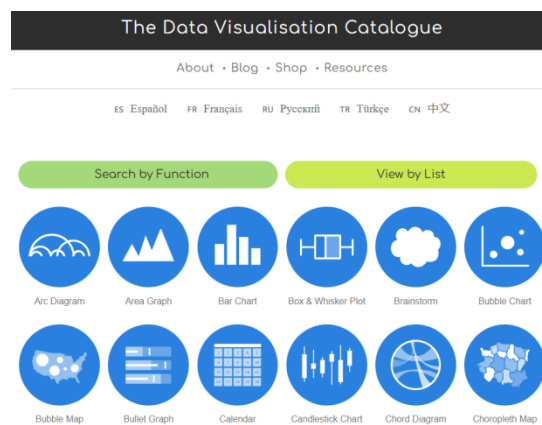


1.3 Herramientas para la elección de gráficos

Aunque existen formatos comunes, frecuentes o populares, no hay que olvidar que el objetivo final es representar la información mediante una imagen, y para esto existen muchas opciones. Para ello se cuenta con múltiples herramientas que son de gran ayuda en la elección de la gráfica que mejor transmite la información que se desea comunicar. Aquí dejamos dos ejemplos para que puedan consultar.



<https://datavizproject.com/>



<https://datavizcatalogue.com/>

2. VISUALIZACIÓN DE DATOS EN PYTHON

2.1 Matplotlib

Matplotlib es una completa biblioteca para crear visualizaciones estáticas, animadas e interactivas en Python. En la documentación web oficial podrás encontrar toda la información sobre esta librería.

En primer lugar hay que instalar matplotlib (en Google Colab no es necesario este paso):

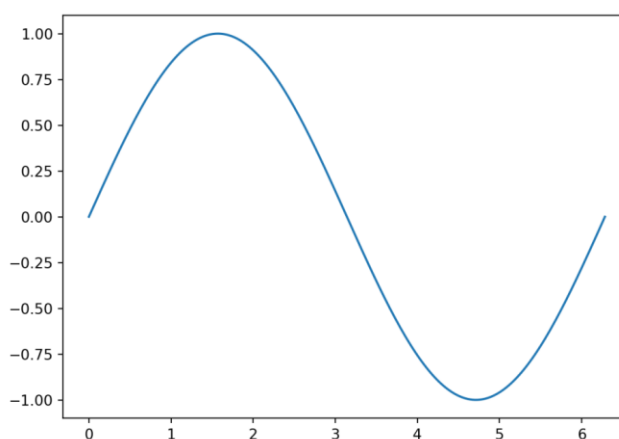
```
pip install matplotlib
```

El siguiente código debería generar una gráfica de prueba. Para ejecutarla, requiere también la librería Numpy. Si no lo tenés, deberás instalarlo con `pip install numpy`.

```
import matplotlib.pyplot as plt
import numpy as np

x = np.linspace(0, 2 * np.pi, 200)
y = np.sin(x)

fig, ax = plt.subplots()
ax.plot(x, y)
plt.show()
```



Matplotlib organiza sus visualizaciones en tres niveles anidados. La figura (Figure) es el lienzo completo, es la ventana o imagen que finalmente se guarda o se muestra. Dentro de ella se ubican uno o más gráficos (Axes) que son las regiones rectangulares donde efectivamente se dibujan los datos. Y cada gráfico tiene sus propios ejes (Axis), habitualmente dos, que son las rectas graduadas que fijan la escala de cada dimensión.

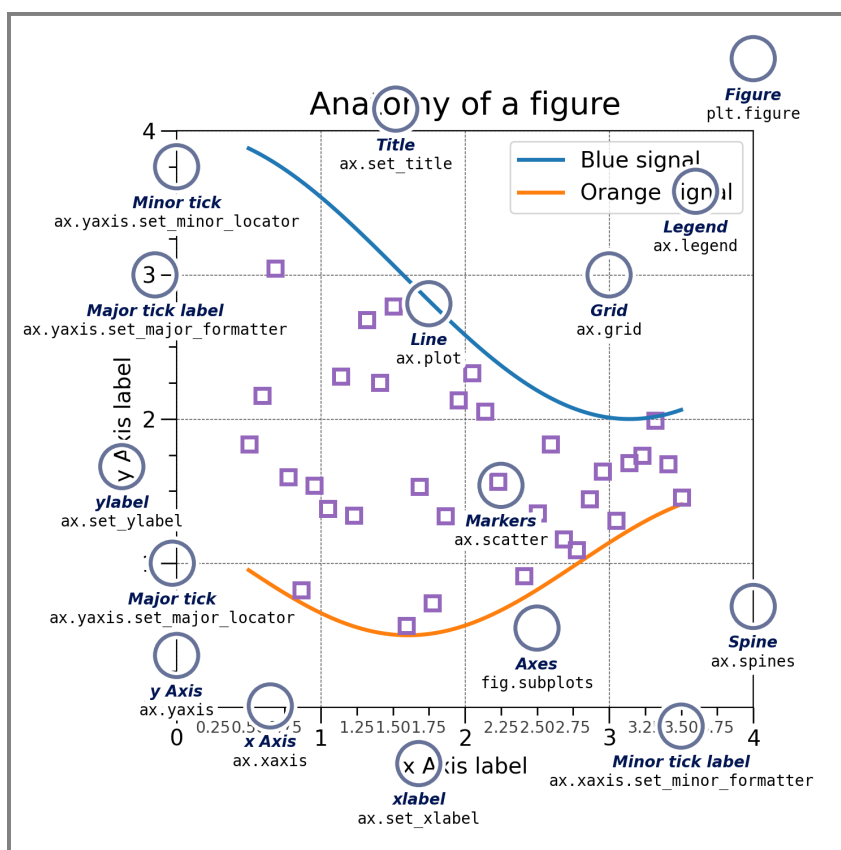
Conviene detenerse en esta distinción porque es la fuente de confusión más común al aprender la librería. En inglés, Axes y Axis se parecen, pero Axes no significa "ejes". Un Axes es un gráfico completo —con su área de dibujo, su título y sus dos ejes adentro—, mientras que un Axis es una sola de esas rectas graduadas. Una figura con cuatro Axes contiene cuatro gráficos, y en total ocho Axis.

En este documento usaremos siempre figura para Figure, gráfico para Axes y eje para Axis.

La forma más sencilla de crear una figura junto con sus gráficos es la función `pyplot.subplots()`.

2.1.1 Partes de una figura

En la siguiente figura se muestran los componentes de una figura de Matplotlib.

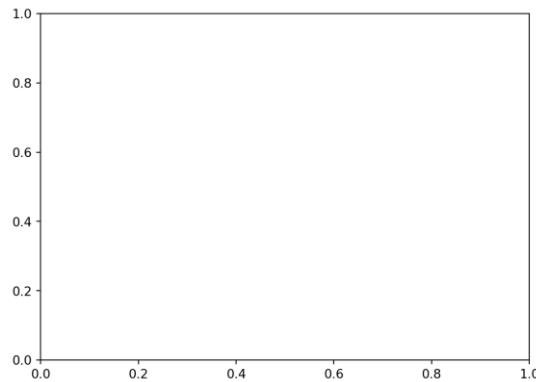


2.1.2 Figura (Figure)

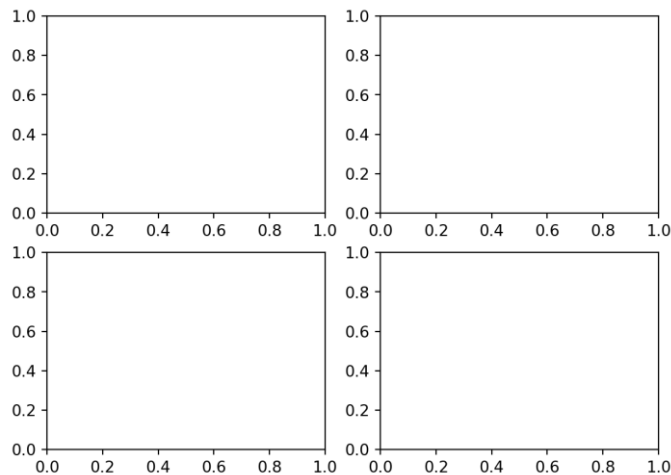
La figura es el contenedor de más alto nivel: gestiona todos los gráficos que contiene, junto con los elementos que pertenecen al conjunto y no a un gráfico en particular, como el título general, la leyenda o las barras de color. También puede alojar subfiguras anidadas.

Se puede crear una nueva figura mediante cualquiera de las siguientes funciones:

```
# Una figura vacía sin gráficos definidos (Axes)
fig = plt.figure()
# Una figura con un gráfico (Axes). Se obtiene el mismo resultado.
fig, ax = plt.subplots()
```



```
# Una figura con una grilla 2x2 con 4 gráficos (Axes)
fig, axs = plt.subplots(2, 2)
```



2.1.3 Gráfico (Axes)

Un gráfico es la región de la figura donde se representan los datos. Es el objeto con el que más se trabaja porque casi toda la construcción de una visualización ocurre a través de sus métodos: agregar datos, definir las escalas y los límites, y colocar los textos.

Cada gráfico contiene dos objetos Axis, el horizontal y el vertical, que establecen las escalas y aportan las marcas y las etiquetas. Además tiene un título propio, que se define con `set_title()`, y una etiqueta para cada eje, que se definen con `set_xlabel()` y `set_ylabel()`.

Una figura puede contener un solo gráfico o varios dispuestos en una grilla, como se vio en los ejemplos anteriores.

2.1.4 Eje (Axis)

El eje es la recta graduada que fija la escala de una dimensión del gráfico. Define el rango de valores representables y genera las marcas (ticks), es decir las líneas cortas que señalan las posiciones de referencia, junto con sus etiquetas numéricas.

La ubicación de las marcas la determina un objeto Locator y el formato de las etiquetas lo define un objeto Formatter. Un gráfico bidimensional tiene dos ejes: xaxis e yaxis.

2.1.5 La estética (Artist)

Todo lo que resulta visible dentro de una figura es un Artist: las líneas, los puntos, las barras, los textos, las leyendas, las marcas de los ejes, e incluso la figura y los gráficos mismos. Es la clase de la que derivan todos los componentes dibujables.

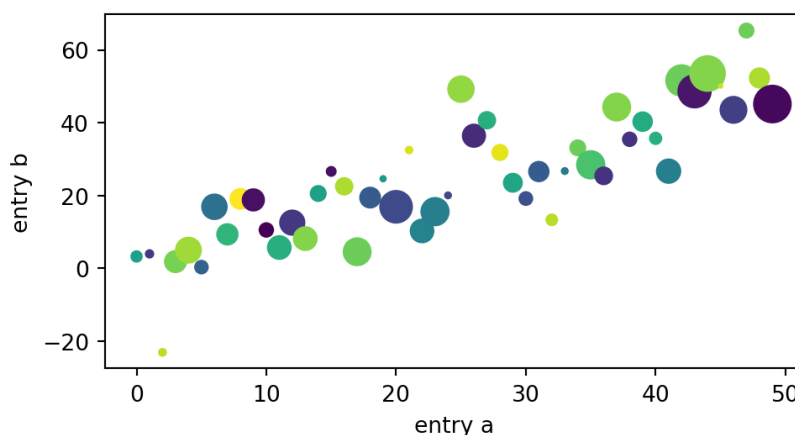
En la práctica esto significa que cualquier elemento que se vea en pantalla puede recuperarse y modificarse por separado —cambiarle el color, el grosor o el tamaño de letra— sin volver a construir el gráfico completo.

2.1.6 Tipos de datos de entrada

Las funciones de trazado esperan como entrada un objeto *numpy.array* o cualquier otro objeto que se puedan pasar a *numpy.asarray*.

```
np.random.seed(196) # seed the random number generator.
data = {'a': np.arange(50),
        'c': np.random.randint(0, 50, 50),
        'd': np.random.randn(50)}
data['b'] = data['a'] + 10 * np.random.randn(50)
data['d'] = np.abs(data['d']) * 100

fig, ax = plt.subplots(figsize=(5, 2.7), layout='constrained')
ax.scatter('a', 'b', c='c', s='d', data=data)
ax.set_xlabel('entry a')
ax.set_ylabel('entry b')
```



NOTA

Para objetos pandas como Dataframes y Series pueden pasarse como arrays, pero puede presentar errores. Para resolver esta cuestión, veremos en el próximo capítulo la librería Seaborn.

2.1.7 Construcción de una figura

1. **Importar librería.** Para este curso será suficiente importar la clase *pyplot*. Por convención, la comunidad nombra esta clase con el alias *plt*.

```
import matplotlib.pyplot as plt
```

2. **Preparar los datos a utilizar.** Para este ejemplo, generaremos dos arrays mediante la librería Numpy.

```
data = {'a': np.arange(50),  
        'b': np.random.randint(0, 50, 50)}  
data['c'] = data['a'] + 10 * np.random.randn(50)  
  
data
```

3. **Crear la figura y los ejes.** Aquí se puede definir el tamaño de la figura y la cantidad de gráficos que contendrá.

```
# Cantidad de gráficos: 1 x 1  
# Tamaño de figura: 8 x 4  
fig, ax = plt.subplots(figsize=(6,4))
```

4. **Trazar los datos.** Aquí se deberá elegir el tipo de gráfico para la visualización de los datos.

```
# Construcción de un gráfico de línea  
ax.plot(data['a'], data['c'], label='Onda Senoidal', color='teal',  
linewidth=2)
```

5. **Personalizar la estética.** En este paso se debe agregar título al gráfico, definir nombres de los ejes, establecer diseño del fondo (color, grilla,), etc.

```
ax.set_title("Análisis de Oscilación")  
ax.set_xlabel("Tiempo (s)")  
ax.set_ylabel("Amplitud")  
ax.grid(True, linestyle='--', alpha=0.6)
```

6. **Añadir la leyenda.** Si tu gráfico lo requiere puedes añadirla con un simple comando.

```
ax.legend()
```

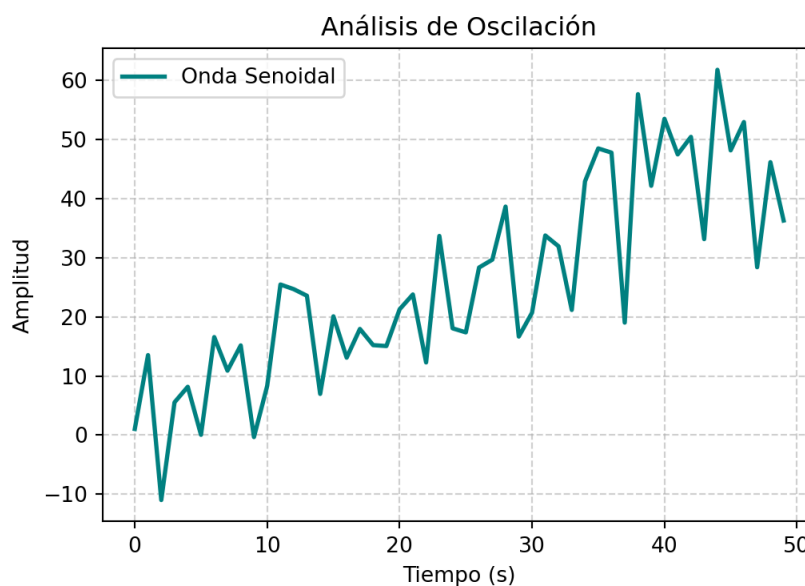
7. **Mostrar el resultado.** Al finalizar deberás colocar la función `plt.show()` para mostrar el resultado final.

```
plt.show()
```

El script de la construcción de la figura de nuestro ejemplo se ve así:

```
# Importación de la librería  
import matplotlib.pyplot as plt  
  
# Preparación de los datos  
data = {'a': np.arange(50),  
        'b': np.random.randint(0, 50, 50)}  
data['c'] = data['a'] + 10 * np.random.randn(50)  
  
# Creación de la figura. Tamaño 8 x 4.  
fig, ax = plt.subplots(figsize=(6,4))  
  
# Construcción de un gráfico de línea  
ax.plot(data['a'], data['c'], label='Onda Senoidal', color='teal',  
linewidth=2)  
  
# Personalización de la estética  
ax.set_title("Análisis de Oscilación")  
ax.set_xlabel("Tiempo (s)")  
ax.set_ylabel("Amplitud")  
ax.grid(True, linestyle='--', alpha=0.6)  
ax.legend()
```

```
# Visualización  
plt.show()
```



2.2 Seaborn

Seaborn es una biblioteca para crear gráficos estadísticos en Python. Proporciona una interfaz de alto nivel para matplotlib y se integra estrechamente con las estructuras de datos de pandas. Asigna automáticamente los valores de los datos a atributos visuales como color, tamaño o estilo, calcula internamente transformaciones estadísticas y decora el gráfico con etiquetas de eje informativas y una leyenda. Al producir gráficos completos a partir de una única llamada a función con argumentos mínimos, seaborn facilita la creación rápida de prototipos y el análisis exploratorio de datos. Y al ofrecer amplias opciones de personalización, además de exponer los objetos matplotlib subyacentes, se puede utilizar para crear figuras pulidas con calidad de publicación.

Recuerde instalar primero la librería. Si estás usando Google Colab, no es necesario este paso.

```
pip install seaborn
```

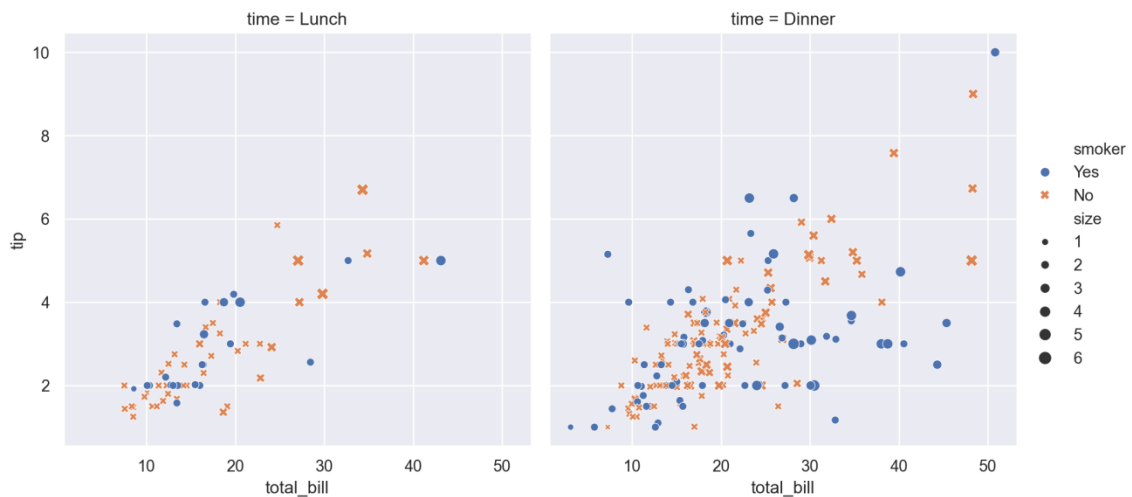
Hecho esto, podrás ejecutar el siguiente código a modo de ejemplo:

```
# Import seaborn
import seaborn as sns

# Apply the default theme
sns.set_theme()

# Load an example dataset
tips = sns.load_dataset("tips")

# Create a visualization
sns.relplot(
    data=tips,
    x="total_bill", y="tip", col="time",
    hue="smoker", style="smoker", size="size",
)
```



La ventaja que presenta Seaborn es que genera gráficos a partir de Dataframe. Por este motivo, esta herramienta será la que utilizaremos en la materia para la visualización de datos.

2.2.1 Construcción de un gráfico

1. **Importar la librería.** Por convención, se nombra la librería con el alias *sns*.

```
import seaborn as sns
```

2. **Preparar los datos en un Dataframe o Serie.** En general, nos referimos al set de datos que han sido pre-procesados anteriormente. En este caso usaremos el dataset "tips", uno de los tantos dataset que ya vienen por defecto en la librería. Para conocer más sobre los dataset de Seaborn, consulte el listado de set de datos disponibles

(<https://github.com/mwaskom/seaborn-data>).

```
tips = sns.load_dataset("tips")
tips
```

3. **Crear la figura y los ejes.** Este paso es el mismo que vimos anteriormente con Matplotlib.

```
# Cantidad de gráficos: 1 x 1
# Tamaño de figura: 8 x 4
fig, ax = plt.subplots(figsize=(6,4))
```

4. **Elegir un estilo.** Seaborn ofrece un conjunto de estilos pre-configurados para hacer gráficos más estéticos. En su página oficial, explican cómo definir diferentes temas o estilos para las gráficas. Actualmente existen 5 estilos distintos: darkgrid, whitegrid, dark, white y ticks. En caso de no indicar ninguno de ellos, se tomará el estilo por default darkgrid.

```
sns.set_theme()
```

5. **Generar la gráfica.** Para eso, debemos llamar a la función de la gráfica requerida. Más tarde veremos algunas de las gráficas disponibles y como configurarlas. Mientras tanto, en este ejemplo haremos un gráfico de línea.

```
sns.barplot(data=tips, x="day", y="tip", hue="smoker",
palette="viridis", capsize=.1, ax=ax)
```

6. **Personalizar la gráfica.** Aquí debemos definir títulos, etiquetas, diseños especiales, etc.

```
plt.title("Promedio de Propinas por Día (Fumadores vs No Fumadores)", fontsize=14)
plt.xlabel("Día de la Semana", fontsize=12)
plt.ylabel("Propina Promedio ($)", fontsize=12)
plt.legend(title="Fumador")
```

7. **Mostrar el resultado.** Como con Matplotlib, hay que ejecutar la función `plt.show()` para poder visualizar la gráfica.

```
plt.show()
```

El script de la construcción de la figura de nuestro ejemplo se ve así:

```
import seaborn as sns

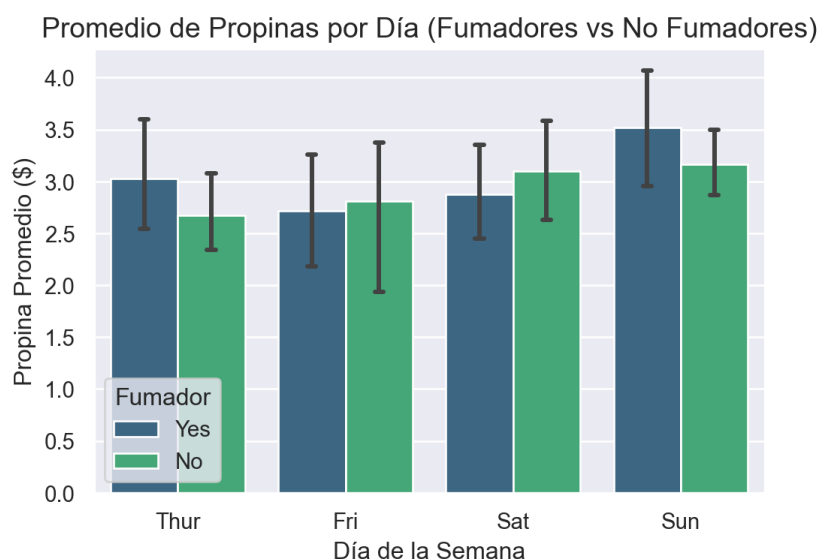
sns.set_theme()

tips = sns.load_dataset("tips")

fig, ax = plt.subplots(figsize=(6,4))
sns.barplot(data=tips, x="day", y="tip", hue="smoker",
palette="viridis", capsize=.1, ax=ax)

plt.title("Promedio de Propinas por Día (Fumadores vs No
Fumadores)", fontsize=14)
plt.xlabel("Día de la Semana", fontsize=12)
plt.ylabel("Propina Promedio ($)", fontsize=12)
plt.legend(title="Fumador")

plt.show()
```



2.2.2 Gráficas en Seaborn

En el ejemplo anterior hicimos un gráfico de barra, pero existen muchas más gráficas.

Función	Descripción
barplot	Gráfico de barra
boxplot	Diagrama de caja
violinplot	Diagrama de violín
countplot	Gráfico de barra (frecuencias)
histplot	Histograma

Función	Descripción
kdeplot	Gráfico de densidad de kernel
scatterplot	Gráfico de dispersión
lineplot	Gráfico de línea
pairplot	Matriz de gráficas de dispersión
heatmap	Mapa de calor
jointplot	Diagrama de dispersión + histograma

Además, en cualquier función de graficado hay que pasar algunos parámetros básicos. A continuación, dejamos el detalle de las más importantes:

Parámetro	Descripción
data	Dataframe de Pandas que contiene la información.
x	Nombre de la columna para el eje horizontal.
y	Nombre de la columna para el eje vertical
ax	Axes del subplot en el cual se construirá la gráfica.
order	Lista con el orden específico en que deben aparecer las categorías
hue	Agrupar los datos por columna y les asigna colores distintos
style	Define el estilo del trazo o la forma del marcador
size	Modifica el grosor de las líneas o el tamaño de los puntos según una variable
palette	Define el mapa de colores a usar
alpha	Define transparencia (de 0 a 1).
estimator	Define la función estadística, como mean, sem, median, max, min, entre otros.
errorbar	Controla las líneas o sombras del error.

3. VISUALIZACIÓN DE DATOS CON R Y GGPLOT

3.1 Ggplot

ggplot2 es el estándar de facto para la visualización de datos en el lenguaje R. Basado en la "**Gramática de los Gráficos**", propone que cualquier visualización puede entenderse como la combinación de capas independientes: datos, sistemas de coordenadas y objetos geométricos. Esto se hará más evidente cuando entremos al detalle de cómo construir un gráfico.

A diferencia del enfoque imperativo de otras librerías, en *ggplot2* se construye el gráfico sumando capas mediante el operador "+".

3.1.1 Construcción de un gráfico

Construir un gráfico en R usando *ggplot2* es como pintar las distintas capas por separado.

1. **Importar la librería.**

```
library(ggplot2)
```

2. **Preparar los datos en un Dataframe o Serie.**

```
data(tips)
```

3. **Crear la figura con los datos.** La función principal es `ggplot()`. Aquí definimos de dónde vienen los datos y qué variables "mapeamos" a los ejes.

```
# Definimos el objeto base usando el dataset tips  
p <- ggplot(data = tips, mapping = aes(x = total_bill, y = tip))
```

"data=tips" especifica el DataFrame y **"mapping=aes(...)"** define la función `aes` (aesthetics) determinando cuáles datos visualizaremos en la gráfica y en cuáles ejes o elementos preatentivos asignaremos cada columna o variable. Aquí le decimos a R "Toma la columna `total_bill` y ponla en el eje X, y la columna `tip` en el eje Y".

4. **Indicar la geometría.** Para ver los datos, debemos sumar (por medio del símbolo `+`) una capa geométrica. Acá es donde indicamos cuál gráfico dibujar:

```
p <- p + geom_point(aes(color = day), alpha = 0.7)
```

`geom_point()` indica que queremos un gráfico de dispersión, `aes(color = day)` incluye al elemento `color` para representar la variable mencionada -esto crea automáticamente una leyenda de colores- y `alpha=0.7` para asignar la transparencia de los puntos

5. **Etiquetas y Anotaciones.** Añadimos la capa de comunicación para que el gráfico sea entendible.

```
p <- p + labs(  
  title = "Relación entre Total de la Cuenta y Propina",  
  subtitle = "Desglosado por día de la semana",  
  x = "Total de la Factura (USD)",  
  y = "Propina (USD)",  
  color = "Día"
```

```
caption = "Fuente: Dataset Tips"
)
```

6. **Indicar estética general de la figura.** Ajustamos el diseño visual del entorno.

```
p <- p + theme_minimal()
```

En un solo bloque de código, la generación del gráfico de dispersión con una línea de tendencia quedaría de la siguiente manera:

```
# Cargar librería
library(ggplot2)

# Generación del gráfico literal
ggplot(data = tips, mapping = aes(x = total_bill, y = tip)) +
  geom_point(aes(color = sex), alpha = 0.6) +
  geom_smooth(method = "lm", color = "black") +
  labs(
    title = "Análisis de Propinas por Género",
    x = "Total de la Cuenta",
    y = "Propina",
    color = "Género"
  ) +
  theme_light()
```

3.1.2 Funciones de graficado (Geoms)

En ggplot2, el tipo de gráfico se define mediante funciones que comienzan con el prefijo `geom_`. En la siguiente tabla se resumen las más utilizadas:

Función	Tipo de Gráfico
<code>geom_point()</code>	Gráfico de dispersión (Scatter plot)
<code>geom_line()</code>	Gráfico de líneas (Series temporales)
<code>geom_bar()</code>	Gráfico de barras (Realiza conteos automáticos)
<code>geom_col()</code>	Gráfico de barras (Usa valores numéricos directos)
<code>geom_boxplot()</code>	Diagrama de caja y bigotes
<code>geom_violin()</code>	Diagrama de violín
<code>geom_histogram()</code>	Histograma de frecuencias
<code>geom_density()</code>	Gráfico de densidad (suavizado)
<code>geom_smooth()</code>	Línea de tendencia o regresión

3.1.3 Parámetros básicos en `aes()`

Dentro de la función `aes()` (aesthetics), vinculamos las columnas de nuestros datos con los atributos visuales de la geometría elegida:

Parámetro	Descripción
x	Variable para el eje horizontal.
y	Variable para el eje vertical.
color	Asigna colores a puntos o líneas según una categoría.
fill	Asigna colores de relleno a áreas (barras, cajas, histogramas).
size	Modifica el tamaño de los elementos según una variable numérica.
shape	Cambia la forma de los puntos según una categoría.
alpha	Define la transparencia de los elementos (de 0 a 1).
group	Define grupos para trazar líneas o áreas independientes

3.1.4 Facetas (Facets)

Las facetas son una de las herramientas más potentes de ggplot2. Permiten dividir un gráfico en múltiples sub-gráficos basados en una o más variables categóricas. Esto facilita la comparación de grupos sin saturar un solo lienzo con demasiados colores o puntos superpuestos.

- `facet_wrap()`: Divide el gráfico en una cuadrícula de paneles según una variable. R decide cuántas filas y columnas usar automáticamente.
- `facet_grid()`: Crea una matriz de gráficos basada en el cruce de dos variables (filas x columnas).

```
# Divide el gráfico anterior en paneles según si es fumador o no  
p + facet_wrap(~ smoker)
```